UTN FRLP

Desarrollo de Software 2026

Introducción al Testing

Clase #21 - 28 de septiembre de 2026

Matías Corti - Martina García Améndola



Dónde quedamos

En C20 cerramos con esta deuda. Qué pasa con ella hoy:

Pendiente	**Lo resuelve**
¿Cómo se prueba PrivateRoute?	← HOY · testing de componentes
La matriz de permisos se verificó a mano, doce veces, con api.http	← HOY · la matriz como suite de integración
El contrato front↔back se rompe en runtime y nadie avisa	← HOY · tests de integración
"Anda en mi máquina"	← HOY · los tests corren igual en todas
HTTPS, dominio real, CORS de producción	C23 · Deploy

Temas de hoy

- Sorteo de fechas de presentación
- Qué es un test automatizado y qué es una regresión
- La pirámide: unitario, integración, E2E
- Anatomía de un test:
  Dado / Cuando / Entonces, `describe` / `it` / `expect`
- Vitest: el runner que usamos y por qué
- Mocks: reemplazar lo que no querés ejecutar
- Testeabilidad: el código que no se puede testear te está diciendo algo
- Integración con supertest: la matriz de permisos, automatizada
- TDD: escribir el test antes
- Componentes React con Testing Library
- El examen del 5/10



Sorteo de presentaciones

Tres fechas, once grupos, una sola mezcla:

Fecha	Cupos	Posiciones del sorteo
C24 · 26/10/26	4 grupos	1 · 2 · 3 · 4
C25 · 02/11/26	4 grupos	5 · 6 · 7 · 8
C26 · 16/11/26	3 grupos	9 · 10 · 11

- La posición define el día **y el orden de exposición** dentro del día
- ~20 min por grupo: demo en vivo + preguntas individuales

Resultado:

C24 (26/10/26)

1 G07
2 G04
3 G05
4 G09

C25 (02/11/26)

5 G02
6 G10
7 G08
8 G11

C26 (16/11/26)

9 G06
10 G01
11 G03

ATENCIÓN

Por el feriado decretado el **09/11/2025** se **modificaron las fechas de presentación**

🔁

Todo lo que probaron a mano era un test

Desde C15 vienen probando:

Clase	Cómo probaron	Cuántas veces
C15–C16	Postman / api.http contra cada endpoint	Cuando lo escribieron
C18	La tabla de doce casos de la matriz de permisos	Una vez
C19	El front contra la API real, mirando la consola	Cada vez que algo se rompía
C20	La matriz desde el navegador: invitado, CLIENTE, ADMIN	Una vez por rol

Eso **es** testing. El problema no es que no prueben: es que lo prueban **ustedes**, **una vez**, y después no.

Hoy lo escribimos una vez y lo corre la máquina, cada vez, en segundos.



¿Qué es un test automatizado?

¿Qué es?

- Código que ejecuta tu código con una entrada conocida y compara el resultado con el esperado
- Corre solo, siempre igual, sin que nadie mire

¿Por qué?

- Un test no prueba que el código funciona: prueba que **sigue** funcionando después de que lo tocaste
- Eso se llama proteger contra la **regresión**: algo que andaba y dejó de andar por un cambio en otro lado
- Sin tests, cada cambio es una apuesta. Con tests, cada cambio tiene una red

Sin tests	**Con tests**
Probar a mano, 20 minutos	npm test, algunos segundos
Probar lo que te acordás	Probar todo, siempre
Miedo a tocar código que anda	Confianza para refactorizar
El bug lo encuentra el cliente	El bug lo encuentra la terminal



Cuánto cuesta un bug según cuándo aparece

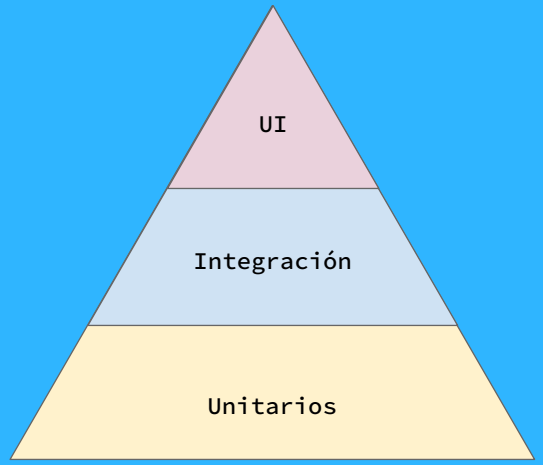
Lo encontrás	Costo	Cómo se ve
Mientras codeas	×1	Un test rojo en la terminal
En la demo o la entrega	×10	"Ayer andaba" delante del cliente
En producción	×100	Un usuario que no puede hacer lo que pagó
Bug de seguridad	×1000+	Datos ajenos, reputación, abogados

Lo que pasa en cualquier proyecto real (posiblemente les pasó):

- "Agregué el rol y rompí el login"
- "Cambié el body del POST y el front dejó de andar"
- "En mi máquina funciona"

Cada uno es un test que no existía.

🔺 TIPOS DE TESTING



← Pocos, lentos, caros
Tests End-to-End (E2E)
Simulan usuarios reales

← Algunos, medianos
Tests de Integración
APIs, BD, componentes

← Muchos, rápidos, baratos
Tests Unitarios
Funciones, métodos

Nivel	Qué aísla	Ejemplo en la Librería	Cantidad	Velocidad
Unitario	Una función, sin nada afuera	authorize("ADMIN") con un CLIENTE → 403	Muchos	ms
Integración	Varias piezas juntas	POST /api/libros sin token → 401, pasando por Express	Algunos	ms a s
E2E	La app completa, como un usuario	Abrir el navegador, loguearse, crear un libro	Pocos	s a min

A qué apunta el TP: base sólida de unitarios + integración de los endpoints que importan. E2E: concepto, no práctica.



TIPOS DE TESTING

🔹 **Tests Unitarios (Unit Tests)**

Qué prueban: UNA función o componente aislado

Ejemplos:

- Función calcularPrecioConDescuento(100, 20) → devuelve 80
- Componente BookCard muestra título y autor correctamente
- Validación de email retorna true/false correctamente

Velocidad: Muy rápidos (milisegundos)

🔹 **Tests de Integración (Integration Tests)**

Qué prueban: VARIAS piezas funcionando juntas

Ejemplo:

- GET /api/books devuelve libros desde la base de datos
- Frontend llama a API y muestra datos en pantalla
- Login completo: form → API → BD → token → redirect

Velocidad: Medianos (segundos)

🔹 **Tests E2E / End-to-End (No lo haremos en el TP)**

Qué prueban: Flujo COMPLETO como usuario real

Ejemplo:

- Abrir navegador → buscar libro → agregar carrito → comprar

Velocidad: Lentos (minutos)

Herramientas: Cypress, Playwright, Selenium



Unitario vs integración: la diferencia que importa

	Unitario	**Integración**
Qué ejecuta	Una función	Un flujo entre capas (ruta → middleware → controller)
Qué necesita de afuera	Nada. Lo que falta, se reemplaza (mock)	Lo real: Express, a veces la DB
Si falla, te dice	Qué función está mal	Que algo entre dos piezas no encaja
Qué no puede ver	Que las piezas encajen	Qué pieza exacta falló
En TP de la Librería	schemas Zod, authorize, formatearPrecio	request(app) .post("/api/libros")

Los dos hacen falta. Uno no reemplaza al otro.

🔁

Mini-cierre: por qué testear

Concepto	En una línea
Test automatizado	Código que ejecuta tu código y compara el resultado con el esperado, solo
Regresión	Algo que andaba y dejó de andar por un cambio en otro lado. Los tests la atrapan
Pirámide	Muchos unitarios, algunos de integración, pocos E2E
Unitario	Una función aislada; lo de afuera se reemplaza
Integración	Varias piezas reales juntas; el flujo, no la pieza
E2E	La app completa como un usuario. Concepto en la materia

✅

Ya saben escribir un test: Dado / Cuando / Entonces

En C05, los criterios de aceptación del backlog:

Dado que estoy en el catálogo, **cuando** escribo un título en el buscador, **entonces** se muestran los libros que coinciden.

Un test es exactamente eso, en código:

Criterio de aceptación	**Test**	**Qué hace**
Dado (contexto)	Arrange (preparar)	Armar los datos, el token, el mock
Cuando (acción)	Act (ejecutar)	Llamar la función, hacer el request, renderizar
Entonces (resultado)	Assert (verificar)	expect(...)

- Se llama **AAA** (Arrange · Act · Assert)
- Cada criterio de aceptación de una historia puede ser uno o más tests
- Si no podés escribir el Dado/Cuando/Entonces, todavía no sabés qué estás testeando



Ejemplo conceptual: login con contraseña incorrecta

Criterio:

Dado un usuario registrado,
cuando intenta loguearse con la contraseña incorrecta,
entonces recibe 401 y el mismo mensaje que si el mail no existiera.

Paso	**Acciones**	**Verificaciones**
Arrange	Existe cliente@libreria.test (seed)	
Act	POST /api/auth/login con { email: "cliente@libreria.test", password: "Incorrecta1" }	
Assert		status = 401 body = { error: "Credenciales inválidas" }

El mismo test se puede escribir de muchas formas. La lógica es siempre esta.



Anatomía en código: describe/it/expect

backend/src/validations/libro.validation.test.ts

```
import { describe, it, expect } from "vitest";
import { libroCreateSchema } from "./libro.validation";

describe("libroCreateSchema", () => {                    // agrupa tests de una misma cosa
  it("rechaza precio negativo", () => {                  // UN comportamiento, en castellano
    const libro = { titulo: "Rayuela", precio: -5, imagen: "x", autorId: 1 };    // Dado
    const resultado = libroCreateSchema.safeParse(libro);        // Cuando
    expect(resultado.success).toBe(false);                       // Entonces
  });
});
```

- **describe**: qué unidad estás probando (una función, un endpoint, un componente)
- **it** (o test): **un** comportamiento.
  El nombre completa la frase "it... rechaza precio negativo"
- **expect**(valor).**matcher**(esperado): la verificación.
  Si falla, el test es rojo y te dice qué esperaba y qué recibió
- Un test puede tener varios expect si verifican **el mismo** comportamiento

Nombres que sirven:

- ❌ it("funciona")
- ✅ it("devuelve 403 a un CLIENTE cuando la ruta pide ADMIN")

Si el test falla, el nombre **tiene que decirte** qué se rompió.

✔ Assertions: las que van a usar

Quiero verificar	Matcher
Igualdad exacta (primitivos)	`expect(x).toBe(42)`
Igualdad profunda (objetos, arrays)	`expect(obj).toEqual({ id: 1 })` · `toStrictEqual`
Que tenga al menos estas propiedades	`expect(obj).toMatchObject({ rol: "ADMIN" })`
Verdadero / falso / null	`toBeTruthy()` · `toBeFalsy()` · `toBeNull()`
Números	`toBeGreaterThan(0)` · `toBeLessThan(100)`
Arrays	`toContain(item)` · `toContainEqual({...})` · `toHaveLength(3)`
Propiedades	`toHaveProperty("token")` · `.not.toHaveProperty("passwordHash")`
Cualquier valor de un tipo	`expect.any(String)` · `expect.any(Number)`
Que tire error	`expect(() => f()).toThrow()`
Mocks	`toHaveBeenCalled()` · `toHaveBeenCalledWith(403)` · `toHaveBeenCalledTimes(1)`
DOM (Testing Library)	`toBeInTheDocument()` · `toBeDisabled()` · `toHaveTextContent("Salir")`

Y siempre existe **.not**: `expect(next).not.toHaveBeenCalled()`.

⚡ Vitest: el runner que usamos

Tres cosas distintas que suelen venir juntas:

Pieza	Qué hace	Acá
Test runner	Encuentra los *.test.ts, los ejecuta, informa rojo/verde	Vitest
Librería de assertions	expect(...).toBe(...)	Viene con Vitest
Testing Library	Renderiza componentes y los consulta como un usuario	@testing-library/react

¿Por qué Vitest?

- Usa la misma configuración que Vite: entiende TypeScript, JSX, import.meta.env y los import de imágenes/CSS **sin configurar nada**
- Rápido: reutiliza el pipeline de Vite y corre en paralelo
- Watch mode que re-ejecuta solo lo que cambió
- En el back también sirve: TypeScript sin compilar, sin ts-jest, sin babel

Actividad: diseñá un test, sin código

En grupos del TP – *10 minutos*

1. Elijan **una** funcionalidad de su TP que **ya esté implementada**
2. Escríbanla como criterio de aceptación: **Dado … cuando … entonces …**
3. Tradúzcanla a la tabla: **Arrange** (¿qué tiene que existir antes? usuario, token, dato en la DB), **Act** (¿qué acción exacta ejecutan? función, request, click), **Assert** (¿qué verifican, con qué valor concreto?)
4. Anótenlo donde quieran (Trello, notas, papel). **Ese es el primer test de [AG-C21]**

Cada grupo lo cuenta en voz alta.

🔁

Mini-cierre: anatomía de un test

Concepto	En una línea
AAA Dado/Cuando/Entonces	Arrange (preparar) · Act (ejecutar) · Assert (verificar). El criterio de aceptación, en código
describe / it / expect	Grupo / un comportamiento / verificación
Nombre del test	Completa "it...": dice qué se rompió sin abrir el archivo
Runner vs assertions vs Testing Library	Ejecuta e informa / compara / renderiza y consulta el DOM
Vitest	Runner que ve el código como Vite lo ve.



Corte

10 minutos.

Cuando volvemos:

- Mocks: reemplazar lo que no querés ejecutar
- Testeabilidad: app vs listen
- Integración con supertest y la matriz como suite
- Pueba TDD, componentes, buenas prácticas, coverage
- Y después, **la práctica**



Mocks y spies: reemplazar lo que no querés ejecutar

¿Qué es un mock?

- Una versión falsa y controlada de una dependencia (una función, un módulo, una respuesta)
- `vi.fn()` crea una función que no hace nada pero **anota** cada llamada: con qué argumentos, cuántas veces
- `vi.fn().mockReturnValue(x)` / `.mockResolvedValue(x)`: además devuelve lo que vos digas

¿Para qué?

- **Aislar:** probar authorize sin levantar Express
- **Verificar interacciones:** "¿llamó a next?", "¿le pasó 403 a status?"
- **Simular lo difícil:** un servicio externo caído, un email que no queremos mandar, un error de red

Qué NO probar con mocks:

- Que las piezas encajen entre sí: si mockeás todo, el test pasa aunque la app no ande
- Un mock que reproduce la lógica de lo que reemplaza: estás testeando el mock

Herramienta	Qué hace
`vi.fn()`	Función falsa que registra llamadas
`vi.spyOn(obj, "metodo")`	Observa (y opcionalmente reemplaza) un método real
`vi.mock("./modulo")`	Reemplaza un módulo entero para el archivo de test

Testeabilidad: el código que no se puede testear te está diciendo algo

Así estaba index.ts hasta hoy:

```
const app = express();
app.use(...)                    // arma la app
app.listen(3000, ...)           // Y la pone a escuchar, en el mismo archivo
```

Para testear POST /api/libros necesito app. Si lo importo, **abre el puerto 3000**. Dos tests en paralelo: "❌ puerto ocupado".

Separar armar de arrancar:

Archivo	**Responsabilidad**	**Quién lo usa**
src/app.ts	Arma Express: middlewares, rutas, errorHandler. Exporta app	Los tests y index.ts
src/index.ts	app.listen(PORT)	Solo npm run dev

La regla general: inyectar lo que varía.
Lo que un módulo necesita de afuera (un puerto, un secret, un cliente de DB, el usuario logueado) tiene que poder entrar desde afuera. Si está adentro, no se puede reemplazar en un test.

- authorize era testeable porque recibe req/res/next: le dimos los falsos
- env.ts era testeable porque lee process.env: le dimos otro
- index.ts no era testeable porque hacía dos cosas



Integración con supertest

supertest le pega a app **sin abrir un puerto**: arma el request, lo pasa por Express completo y te devuelve la respuesta.

```
import request from "supertest";
import app from "../app";

const res = await request(app)
  .post("/api/libros")                                  // método + ruta
  .set("Authorization", `Bearer ${token}`)              // headers
  .send({ titulo: "Rayuela", precio: 7000, … });        // body (JSON)

res.status   // 201
res.body     // el JSON de la respuesta, ya parseado
```

- Pasa por **todo**: cors, express.json, authenticate, authorize, validate, el controller, errorHandler
- Es la misma request que hacía api.http, pero con un expect al final y sin nadie mirando
- await siempre: es asíncrono. Sin await, el test termina antes de la respuesta (va en "Rompelo")



La DB en los tests: tres opciones

Opción	Cómo	Cuándo
Sin DB	El request corta antes de llegar a Prisma: 401, 403, 400	Toda la matriz de permisos y validaciones. Core de hoy
La DB de Docker, con seed	docker compose up -d db	Los 200: GET /api/libros, login. Lo que crea, lo borra (afterAll)
Una DB solo para tests	DATABASE_URL de test, seed antes de cada suite, se vacía al final	Proyectos reales. Concepto hoy; en el TP, si llegan tiempo

Por qué la primera opción cubre tanto: el orden authenticate → authorize → validate → controller de C18 hace que un request sin permiso **nunca** toque la DB. Ese orden se decidió por seguridad; hoy además regala tests rápidos.

Un token firmado en el test (con el secret de test) reemplaza al login: no hace falta un usuario real para tener un token de ADMIN.



La matriz de permisos, ahora como suite (sin DB)

La tabla de C18, con dos columnas nuevas. Primero los casos que **no** tocan la base:

Request	Token	Esperado	Archivo
POST /api/libros	-	401 "Falta el token"	libro.routes.test.ts
POST /api/libros	CLIENTE	403	libro.routes.test.ts
POST /api/libros {precio:-5}	ADMIN	400, no 403	libro.routes.test.ts
POST /api/libros	adulterado	401 "Token inválido"	libro.routes.test.ts
DELETE /api/libros/1	CLIENTE	403	libro.routes.test.ts
GET /api/no-existe	-	404	libro.routes.test.ts

Los seis casos de esta tabla se testean sin de llegar a Prisma, no necesitan la base corriendo.



La matriz de permisos, ahora como suite (con DB)

Los casos que sí necesitan la base, contra la DB de Docker:

Request	Token	Esperado	Archivo
POST /auth/login correcto	-	200 con token, sin passwordHash	auth.routes.test.ts
POST /auth/login password mal	-	401 "Credenciales inválidas"	auth.routes.test.ts
POST /auth/login mail inexistente	-	el mismo 401	auth.routes.test.ts
GET /auth/yo	del login	200	auth.routes.test.ts
GET /api/libros	-	200 con autor anidado	libro.routes.db.test.ts
POST /api/libros	ADMIN	201	libro.routes.db.test.ts (extensión)

Para el TP: la matriz de permisos que está en Documentación/ es la lista de tests de integración. Ya la tienen escrita.

🔁

Mini-cierre: mocks, testeabilidad, integración

Concepto	En una línea
Mock / spy	Dependencia falsa y controlada. vi.fn() recuerda cómo la llamaron
Para qué mockear	Aislar, verificar interacciones, simular lo difícil. No para esconder que las piezas no encajan
Testeabilidad	Si testear cuesta una acrobacia, el módulo hace dos cosas o tiene algo enterrado
Inyectar lo que varía	Puerto, secret, DB, usuario: entran desde afuera para poder reemplazarlos
app vs listen	app.ts arma y exporta; index.ts escucha. Los tests importan app
supertest	Request completo por Express sin abrir un puerto. await request(app).post(...)
Sin DB	401/403/400 cortan antes de Prisma gracias al orden de middlewares



TDD

EL TEST PRIMERO

Test-Driven Development: escribir el test **antes** que el código, en un ciclo corto:

Paso	Qué hacés	Qué ves
🔴 Rojo	Escribís un test para algo que todavía no existe	Falla (a veces ni compila)
🟢 Verde	Escribís lo mínimo para que pase	Pasa. Aunque sea feo
🔵 Refactor	Mejorás el código sin cambiar el comportamiento	Sigue verde

Qué garantiza:

- Que el test puede fallar (lo viste rojo). Un test que nunca fue rojo puede estar pasando por casualidad
- Que solo escribís el código que un test pide: sin funciones "por si acaso"
- Que pensás la interfaz (cómo se usa) antes que la implementación

Cuándo sí / cuándo no:

- **Sí:** funciones puras, reglas de negocio, bugs (primero el test que lo reproduce, después el fix)
- **No conviene:** explorar una librería nueva, UI que todavía no sabés cómo va a ser, prototipos que vas a tirar



Testear componentes: comportamiento, no implementación

¿Qué es un test de componente?

- Renderizás el componente en un DOM simulado (jsdom) y lo consultás **como lo vería un usuario**: por texto, por rol, por label
- No mirás el estado interno ni los hooks: si mañana Header cambia useState por otra cosa y se ve igual, el test sigue verde

	Qué hace
`render(<Comp />)`	Monta el componente en jsdom
`screen.getByText('Salir')`	Busca por texto. Falla si no está
`screen.getByRole('link', { name: 'Nuevo libro' })`	Busca por elemento accesible (link, button, heading...)
`screen.queryByText('Nuevo libro')`	Igual, pero devuelve null si no está: para verificar ausencia
`expect(el).toBeInTheDocument()`	Matcher de jest-dom

Un componente tiene dependencias como cualquier módulo. Header usa useNavigate (router), useBusqueda y useAuth (contexts). Para renderizarlo en un test hay que **darle** las tres. Con auth **falso**: eso es un mock.



Buenas prácticas

- **Independientes:** cada test corre solo y en cualquier orden. Vitest los puede correr en paralelo; si uno depende del anterior, un día falla sin que nadie lo entienda
- **beforeEach para preparar, afterAll para limpiar:** el Arrange repetido va ahí. Lo que un test crea en la DB, lo borra
- **Un comportamiento por test:** si falla, el nombre dice qué. "Login OK y devuelve 401 con password mal" son dos tests
- **Comportamiento, no implementación:** verificá lo que sale (status, body, texto en pantalla), no cómo se calculó
- **Datos realistas y simples:** cliente@libreria.test, "Rayuela". No asdf, no test123
- **Los tests son código:** van al repo, pasan por PR, se leen. Un test ilegible es deuda
- **Verde ≠ correcto:** un test que nunca viste rojo puede estar pasando por casualidad. TDD lo resuelve de fábrica; si no, rompé algo a propósito
- **No testees la librería:** que Zod valide emails ya lo probó Zod. Testeá **tu** schema: que el email se normalice, que el precio no sea negativo



Coverage: qué mide y qué no

Qué es: el porcentaje de líneas (o ramas, o funciones) de tu código que **algún test ejecutó.**

Mide	**No mide**
Que la línea se ejecutó	Que alguien verificó el resultado
Qué partes del código nunca corre ningún test	Que los expect sean buenos
Dónde faltan tests, a grandes rasgos	Que el código sea correcto

```
it("cubre login", async () => {
  await request(app).post("/api/auth/login")
      .send({ email: "a@b.c", password: "x" });
});   // 100% de coverage del login. Cero assertions. No prueba nada.
```

- **100% de coverage no significa sin bugs.**
  40% con buenos asserts vale más que 95% sin ellos
- Útil como **mapa**: "nunca testeamos errorHandler para P2002".
  No como **meta**
- Así lo instalamos: **npm install @vitest/coverage-v8**
  Se ejecuta así: **npx vitest run --coverage**

🔁

Mini-cierre: TDD, componentes, prácticas

Concepto	En una línea
TDD	Rojo → verde → refactor. Garantiza que el test puede fallar y que solo escribís lo que un test pide
Test de componente	Renderizar en jsdom y consultar como un usuario: texto, rol, label
Comportamiento, no implementación	Verificá lo que sale, no cómo. Un refactor que no cambia la pantalla no rompe tests
getBy vs queryBy	Falla si no está / devuelve null. El segundo es para verificar ausencia
Independencia	Cada test solo, en cualquier orden. beforeEach prepara, afterAll limpia
Coverage	Mide líneas ejecutadas, no verificadas. 100% ≠ sin bugs



PRÁCTICA



La práctica: once pasos, back y front (1/2)

Branch de partida: C20. Commit después de cada paso: test(c21): paso N. Cada commit deja **todos** los tests verdes.

Paso	Dónde	Archivo	Qué
0	back	package.json	Scripts · docker compose run --rm api npx vitest run → "No test files found"
1	back	validations/libro.validation.test.ts	Unitario puro: el primer verde
2	back	validations/auth.validation.test.ts	Lo que el schema transforma
3	back	middlewares/auth.middleware.test.ts	authorize con vi.fn() → ⚠️ → vitest.config.mts
4	back	mismo archivo	authenticate con tokens reales
5	back	app.ts + index.ts	Separar armar de escuchar

La práctica: once pasos, back y front (2/2)

Paso	Dónde	Archivo	Qué
6	back	routes/libro.routes.test.ts	La matriz sin DB: 401 · 403 · 400
7	back	routes/auth.routes.test.ts	Login 200 / 401 contra la DB de Docker
8	front	vite.config.ts + setupTests.ts	jsdom + matchers
9	front	utils/formatearPrecio.test.ts → .ts	Kata TDD en watch mode
10	front	components/Layout/Header.test.tsx	El Header según el rol

Meta mínima: pasos 0–4 y 6 (22 verdes **sin Docker**) + paso 9. Con eso hay unitario, mock, integración y TDD: los cuatro tipos de [AI-C21].



Instalación: en front y back

```bash
# FRONTEND: en su directorio (sin Docker)
cd frontend
npm install -D vitest jsdom @testing-library/react
@testing-library/jest-dom @vitest/coverage-v8
# tiene que mostrar una versión
npm ls vitest


# BACKEND: en su directorio (CON Docker)
cd ../backend
docker compose exec api npm install -D vitest supertest
@types/supertest @vitest/coverage-v8
# tiene que mostrar una versión
docker compose exec api npm ls vitest
```



Paso 0: el runner sin tests

backend/package.json

```
"scripts": {
  "dev":        "tsx watch src/index.ts",
  "start":      "tsx src/index.ts",
  "test":       "vitest run",
  "test:watch": "vitest"
}
```

```
cd backend
docker compose run --rm api npxvitest run
# No test files found, exiting with code 1
# include: **/*.{test,spec}.?(c|m)[jt]s?(x)
```

- vitest run: corre una vez y termina. Es lo que corre un CI
- vitest (sin run): queda mirando los archivos y re-ejecuta al guardar
- Encuentra solo cualquier *.test.ts: no hay que registrar nada*

Listo cuando: ves "No test files found". Es la salida correcta: el runner anda y no hay tests.

Ejemplo Completo: DS26-Libreria (C21)

Paso 1: el primer verde, sin nada afuera

backend/src/validations/libro.validation.test.ts

```
import { describe, it, expect } from "vitest";
import { libroCreateSchema, idParamSchema } from "./libro.validation";

const libroValido = { titulo: "Rayuela", precio: 7000, imagen:
"https://img/r.jpg", autorId: 1 };

describe("libroCreateSchema", () => {
  it("rechaza precio negativo", () => {
    const r = libroCreateSchema.safeParse({ ...libroValido, precio: -5 });
    expect(r.success).toBe(false);
    if (!r.success) expect(r.error.issues[0].path).toEqual(["precio"]);
  });
  it("recorta espacios del título", () => {
    const r = libroCreateSchema.safeParse({ ...libroValido, titulo: "
Rayuela  " });
    if (r.success) expect(r.data.titulo).toBe("Rayuela");
  });
});
```

- Un schema Zod es una función pura: entra un objeto, sale un resultado. **Cero dependencias**: el test ideal
- libroValido afuera del it: el Arrange que se repite, una sola vez
- El if (!r.success) es para que TypeScript sepa que error existe. Zod 4: error.issues

Listo cuando: docker compose run --rm api npx vitest run → ✓ … (2 tests). Agregá el de idParamSchema con { id: "42" } → data.id es el **número** 42.

Ejemplo Completo: DS26-Libreria (C21)



Paso 2: lo que el schema transforma

backend/src/validations/auth.validation.test.ts

```
describe("loginSchema", () => {
  it("normaliza el email: saca espacios y pasa a minúsculas" , () => {
    const r = loginSchema.safeParse({ email: "  Admin@Libreria.TEST " , password: "x"
});
    expect(r.success).toBe(true);
    if (r.success) expect(r.data.email).toBe("admin@libreria.test" );
  });
});

describe("registroSchema", () => {
  it("una contraseña débil acumula los tres errores" , () => {
    const r = registroSchema.safeParse({ nombre: "Ana", email: "ana@libreria.test" ,
password: "corta" });
    if (!r.success) expect(r.error.issues.map(i => i.message)).toHaveLength(3);
  });
});
```

- Este test documenta una decisión de C18: `.trim().toLowerCase()` **antes** de validar el email
- Si alguien mañana "simplifica" el schema y saca el toLowerCase(), este test se pone rojo. Eso es una regresión atrapada

Listo cuando: 2 archivos, todos verdes.

Ejemplo Completo: DS26-Libreria (C21)



Paso 3: authorize con req, res y next falsos

[Ejemplo Completo: DS26-Libreria (C21)]

backend/src/middlewares/auth.middleware.test.ts

```
import { describe, it, expect, vi } from "vitest";
import type { Request, Response, NextFunction } from "express";
import { authorize } from "./auth.middleware";

function mocks(req: Partial<Request> = {}) {
  const res = { status: vi.fn().mockReturnThis(), json: vi.fn().mockReturnThis() } as
unknown as Response;
  const next = vi.fn() as unknown as NextFunction;
  return { req: req as Request, res, next };
}

it("responde 403 a un CLIENTE cuando la ruta pide ADMIN" , () => {
  const { req, res, next } = mocks({ usuario: { id: 2, rol: "CLIENTE" } });
  authorize("ADMIN")(req, res, next);
  expect(res.status).toHaveBeenCalledWith(403);
  expect(next).not.toHaveBeenCalled();
});
```

- authorize no sabe si res es de Express o un objeto con dos funciones falsas. Solo llama res.status(403).json(...)
- vi.fn() es una función que **recuerda cómo la llamaron**: eso es lo que verificamos
- mockReturnThis(): res.status(403) tiene que devolver res para que .json(...) encadene, como el real

Corrélo. Va a fallar. Siguiente filmina.

⚠️ Si te pasa esto: Falta JWT_SECRET en el .env

[Ejemplo Completo: DS26-Libreria (C21)]

```
FAIL  src/middlewares/auth.middleware.test.ts
Error: Falta JWT_SECRET en el .env
 ❯ obligatorio src/config/env.ts:5:21
```

Qué pasó	Por qué
El test importa auth.middleware.ts, que importa config/env.ts	env.ts tira al importar si falta la variable. Es la barrera de C18 funcionando
En Docker la variable existe	La inyecta Compose con env_file. En el host nadie carga .env
Vitest no lee .env solo	Solo expone variables VITE_*. Las demás hay que dárselas

Solución: un secret solo para tests, en la config del runner:

backend/vitest.config.mts

```ts
import { defineConfig } from "vitest/config";
import { loadEnv } from "vite";

// lee tu .env (expande ${POSTGRES USER} como Compose)
const env = loadEnv("test", process.cwd(), "");

export default defineConfig({
  test: {
    env: {
      JWT_SECRET: "secreto-solo-para-tests",     // nunca el real
      DATABASE_URL: env.DATABASE_URL ?? "",
    },
  },
});
```

Listo cuando: docker compose run --rm api npx vitest run → 3 archivos verdes.



Paso 4: authenticate con tokens de verdad

backend/src/middlewares/auth.middleware.test.ts (continúa)

```
import jwt from "jsonwebtoken";
import { JWT_SECRET } from "../config/env";  // el de test, ya seteado por vitest.config

it("con un token válido llena req.usuario y llama a next", () => {
  const token = jwt.sign({ id: 7, rol: "CLIENTE" }, JWT_SECRET, { expiresIn: "1h" });
  const { req, res, next } = mocks({ headers: { authorization: `Bearer ${token}` } });
  authenticate(req, res, next);
  expect(next).toHaveBeenCalledTimes(1);
  expect(req.usuario).toEqual({ id: 7, rol: "CLIENTE" });
});

it("401 'Token expirado' con un token vencido", () => {
  const token = jwt.sign({ id: 7, rol: "CLIENTE" }, JWT_SECRET, { expiresIn: "-1s" });
  // ...
  expect(res.json).toHaveBeenCalledWith({ error: "Token expirado" });
});
```

- No hay que esperar dos horas para probar el vencimiento: expiresIn: "-1s" fabrica un token ya vencido
- Tercer caso: firmado con "otro-secret" → "Token inválido". Es el caso 14 de api.http de C18, sin editar el token a mano
- Con esto, authenticate y authorize están **completamente** cubiertos sin Express ni DB

Listo cuando: 7 tests en el archivo, 16 en total.

Ejemplo Completo: DS26-Libreria (C21)



Paso 5: app.ts arma, index.ts escucha

backend/src/app.ts

```
const app = express();
app.use(cors(corsOptions));
app.use(express.json());
app.use("/api/auth", authRoutes);
app.use("/api/libros", libroRoutes);
app.use("/api/autores", autorRoutes);
app.use((_req, res) => res.status(404).json({ error: "Ruta no encontrada" }));
app.use(errorHandler);
export default app;
```

backend/src/index.ts

```
import app from "./app";

const PORT = Number(process.env.PORT) || 3000;
app.listen(PORT, () => console.log(`Servidor escuchando en
http://localhost:${PORT}`));
```

backend/src/index.ts queda en tres líneas.

Listo cuando: docker compose up -d y curl localhost:3000/ responde igual que antes.

Ejemplo Completo: DS26-Libreria (C21)



Paso 6: la matriz sin DB

backend/src/routes/libro.routes.test.ts

```
import request from "supertest";
import jwt from "jsonwebtoken";
import app from "../app";
import { JWT_SECRET } from "../config/env";

const tokenCliente = jwt.sign({ id: 2, rol: "CLIENTE" }, JWT_SECRET, { expiresIn:
"1h" });
const tokenAdmin   = jwt.sign({ id: 1, rol: "ADMIN" },   JWT_SECRET, { expiresIn:
"1h" });

it("con token de CLIENTE → 403" , async () => {
  const res = await request(app).post("/api/libros")
    .set("Authorization", `Bearer ${tokenCliente}`).send(libroNuevo);
  expect(res.status).toBe(403);
});

it("ADMIN con body inválido → 400, no 403" , async () => {
  const res = await request(app).post("/api/libros")
    .set("Authorization", `Bearer ${tokenAdmin}`).send({ precio: -5 });
  expect(res.status).toBe(400);
  expect(res.body.detalles).toContainEqual({ campo: "precio", mensaje: "El precio
debe ser mayor a 0" });
});
```

- Los tokens se **firman en el test**: sin login, sin usuario en la DB, con el secret de test
- El tercer test verifica una decisión de diseño de C18 (autorizar antes de validar). Si alguien invierte el orden, se pone rojo
- res.body.detalles: la forma exacta que devuelve errorHandler. **El test fija el contrato**

Listo cuando: 6 tests verdes en el archivo, **22 en total, con Docker apagado.**

Ejemplo Completo: DS26-Libreria (C21)



Paso 7: los 200, contra la DB de Docker

backend/src/routes/auth.routes.test.ts

```
import { prisma } from "../config/prisma";
afterAll(() => prisma.$disconnect());          // cerrar la conexión al terminar

it("credenciales del seed → 200 con token y usuario sin passwordHash", async () => {
  const res = await request(app).post("/api/auth/login")
    .send({ email: "cliente@libreria.test", password: "Cliente1234" });
  expect(res.status).toBe(200);
  expect(res.body.token).toEqual(expect.any(String));
  expect(res.body.usuario).toMatchObject({ email: "cliente@libreria.test", rol:
"CLIENTE" });
  expect(res.body.usuario).not.toHaveProperty("passwordHash");
});

it("mail que no existe → el MISMO 401 (no revela si el mail existe)", async () => {
  const res = await request(app).post("/api/auth/login")
    .send({ email: "nadie@libreria.test", password: "Cliente1234" });
  expect(res.body).toEqual({ error: "Credenciales inválidas" });
});
```

Necesita: docker compose up -d db, seed cargado, .env con DATABASE_URL (el config lo pasa a localhost).

⚠️ **Si te pasa esto:** Can't reach database server at 127.0.0.1:5432 y expected 500 to be 200 → Docker está apagado. Los otros 22 siguen verdes: eso es exactamente para lo que separamos los archivos.

Listo cuando: 27 verdes.

Ejemplo Completo: DS26-Libreria (C21)



Paso 8: Vitest en el front

En frontend/, en el host, como siempre.

frontend/vite.config.ts

```
/// <reference types="vitest/config" />
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',    // los componentes necesitan un DOM: jsdom lo simula
    setupFiles: './src/setupTests.ts', // corre antes de cada archivo de test
  },
})
```

frontend/src/setupTests.ts → una línea: import '@testing-library/jest-dom/vitest';

Scripts en package.json: "test": "vitest run", "test:watch": "vitest".

Listo cuando: npx vitest run → "No test files found".

Ejemplo Completo: DS26-Libreria (C21)



Paso 9: Prueba TDD, formatearPrecio

Hoy LibroCard muestra ${precio} → $4500. Queremos $ 4.500. **Primero el test.**

frontend/src/utils/formatearPrecio.test.ts

```
import { describe, it, expect } from 'vitest';
import { formatearPrecio } from './formatearPrecio';   // todavía no existe

describe('formatearPrecio', () => {
  it('4500 → "$ 4.500"', () => {
    expect(formatearPrecio(4500)).toBe('$ 4.500');
  });
});
```

	En npm run test:watch
🔴	Error: Cannot find module './formatearPrecio'
🟢 mínimo	export const formatearPrecio = (_p: number) => '$ 4.500'; → pasa. Es trampa, y está bien
🔴 otra vez	Segundo test: 1234567 → "$ 1.234.567". La trampa se cae
🟢 de verdad	`$ ${new Intl.NumberFormat('es-AR').format(precio)}`
🔵 refactor	Tercer test: 0 → "$ 0". Sigue verde. LibroCard pasa a usarla

Listo cuando: 3 verdes, y LibroCard.tsx muestra {formatearPrecio(precio)}.

Ejemplo Completo: DS26-Libreria (C21)



Paso 10: el Header según el rol

frontend/src/components/Layout/Header.test.tsx

```
function renderHeader(usuario: Usuario | null) {
  const auth = { usuario, cargando: false, estaAutenticado: usuario !== null,
    tieneRol: (rol: Rol) => usuario?.rol === rol, login: vi.fn(), logout:
vi.fn() };
  return render(
    <MemoryRouter>
      <BusquedaProvider>
        <AuthContext.Provider value={auth}>
          <Header />
        </AuthContext.Provider>
      </BusquedaProvider>
    </MemoryRouter>,
  );
}

it('CLIENTE: saluda por nombre y NO ve "Nuevo libro"', () => {
  renderHeader({ id: 2, email: 'cliente@libreria.test', nombre: 'Cliente',
rol: 'CLIENTE' });
  expect(screen.queryByText('Nuevo libro')).not.toBeInTheDocument();
});
```

Tres tests: sin sesión (Ingresar, sin "Nuevo libro") · CLIENTE · ADMIN (getByRole('link', { name: 'Nuevo libro' })). **Es la columna "Qué hace la UI" de la matriz de C20.**

Ejemplo Completo: DS26-Libreria (C21)



Paso 10: qué puede fallar

⚠️ Si te pasa esto	Es porque
Cannot read properties of undefined (reading 'Provider')	AuthContext no está exportado. Agregar export en AuthContext.tsx: un módulo testeable expone lo que hay que inyectar
ReferenceError: document is not defined	Falta environment: 'jsdom' (paso 8)
useAuth debe usarse dentro de <AuthProvider>	Renderizaste <Header /> pelado. El error de C20 haciendo su trabajo

Listo cuando: 6 verdes en el front, npx tsc -p tsconfig.app.json --noEmit y npm run build limpios.



Rompelo a propósito

Elegí dos o tres según el tiempo y reproducilas.

Qué hacés	Qué ves	Qué aprendés
expect(res.body) .toBe({ error: "Ruta no encontrada" })	Rojo: expected {…} to be {…} // Object.is equality, replace "toBe" with "toStrictEqual"	toBe compara referencias. Objetos: toEqual / toStrictEqual
Sacá el await de request(app)… y dejá el expect adentro de un .then	El test muestra ✓, y abajo Unhandled Rejection: expected 404 to be 200. La corrida termina roja	Un test async sin await termina antes de verificar. Puede pasar mintiendo
Cambiá un it por it.only	Todos los demás: skipped. La suite "pasa" con un test	.only es para depurar. Si lo commiteás, apagaste la suite
En mocks(), sacá .mockReturnThis( ) de status	TypeError: Cannot read properties of undefined (reading 'json')	El mock tiene que comportarse como lo real donde el código lo usa
En el test de login, cambiá la password del seed	Rojo con expected 401 to be 200	El test depende de datos: el seed es parte del Arrange
Corré los tests con Docker apagado	22 verdes, los de DB rojos con Can't reach database server	Por qué los separamos en archivos

⚡

¿Terminaste? Extensiones

Todas en el branch C21:

1. **libro.routes.db.test.ts:** GET /api/libros 200 con autor anidado, ?disponible=false, GET /api/libros/999999 → 404, y POST como ADMIN → 201 con afterAll que borra lo que creó
2. **PrivateRoute con MemoryRouter:** initialEntries={['/libros/nuevo']}, Routes con la ruta protegida y /login, AuthContext.Provider con usuario: null → aparece la página de login; con ADMIN → aparece el Outlet
3. **libroSchema.test.ts del front:** precio: "7000" (string del <input>) → 7000 (número). Unitario puro en el front
4. **Mockear Prisma:** vi.mock("../config/prisma", () => ({ prisma: { libro: { findMany: vi.fn() } } })) y mockResolvedValue([...]) para testear libroService.findAll sin DB
5. **Coverage en casa:** npm i -D @vitest/coverage-v8 y npx vitest run --coverage.



Lo que falta, falta a propósito

Pendiente	Qué es	Por qué hoy no
E2E (Playwright, Cypress)	Un navegador real ejecutando la app	Requiere instalar navegadores. Concepto en la pirámide
Coverage	Qué líneas ejecutan los tests	No está instalado.
Mockear Prisma	Testear services sin DB	Para la Librería alcanza con unitarios puros + integración. Extensión 4
Base de datos de test separada	DATABASE_URL propia, seed por suite	Se nombró el problema y la solución. Proyectos reales
CI (GitHub Actions)	Que los tests corran solos en cada push	Deploy (C23) o fuera de alcance
PrivateRoute con router	La deuda de C20	Extensión 2, con MemoryRouter
Snapshot testing, property-based	Otras técnicas	Fuera de la materia



El examen: 5 de octubre

Cuándo	Lunes 5/10, en el horario de clase. *No hay clase de contenido ese día*
Qué	Examen individual, presencial. Sin computadora, sin apuntes, sin celular
Cubre	C01 a C21: todo lo visto hasta hoy, ambos bloques de front y back
Formato	Duración: 90 minutos Preguntas múltiple choice (temas de teoría y de práctica)
Recuperatorio	C28 (30/11)

Actividades

Actividad Individual

1° instancia hasta el **18/10/2026**
*2° instancia: **15/11/2026** (Cierre Bloque 4)*

[AI-C21] Testing en la Librería

Referencia en el repo de ejemplo branch C21

En ds2026-actividades: copiar el sitio de la librería de la Clase 20 en **ejercicios/c21-testing/**

1. Vitest configurado en backend/ (vitest.config.mts con JWT_SECRET de test y DATABASE_URL a localhost) y en frontend/ (test en vite.config.ts + setupTests.ts)
2. app.ts separado de index.ts; npm run dev en Docker sigue funcionando
3. Scripts test y test:watch en ambos package.json
4. Al menos 5 tests verdes, que incluyan como mínimo: uno unitario puro (un schema Zod), uno con vi.fn() (authorize o authenticate), uno de integración con supertest (un caso de la matriz de permisos), uno de componente con Testing Library (Header por rol)
5. Los tests que necesitan DB, en un archivo aparte, con un comentario que lo diga
6. npx tsc --noEmit (back) y npx tsc -p tsconfig.app.json --noEmit (front) limpios con los tests adentro. Cero dependencias nuevas
7. Commiteá y avisá en #actividades

→ hilo **[AI-C21] Testing en la Librería**

Actividad grupal

Hasta el **18/10/2026**
(inclusive)

[AG-C21] Testing en el TP

1. Vitest configurado en back y front del TP, con scripts test y test:watch
2. Al menos 5 tests verdes. La matriz de permisos de Documentación/ es la suite de integración: cada fila con 401/403 es un test sin DB
3. El test que diseñaron hoy en la actividad (Dado/Cuando/Entonces), implementado
4. README.md con sección "Testing": cómo correrlos (qué hace falta levantar) y qué cubren
5. Los tests que necesitan DB, separados y señalados
6. Repartir el trabajo: al menos un PR con review cruzado por integrante. Los tests se revisan como código
7. Tarjetas de Trello a "Done", retrospectiva del sprint y planning del siguiente
8. Aviso en #actividades
   → hilo **[AG-C21] Testing en el TP**



Próximas fechas

Fecha	Qué
5/10 - C22	Examen individual. Presencial. Cubre C01-C21.
12/10	Feriado. No hay clase
18/10	Cierre 1ª instancia [AI-C21] y [AG-C21]
19/10 - C23	Deploy y entornos. Consolidación y preparación de presentaciones.
26/10 - 2/11 - 16/11	Presentaciones finales (según el sorteo de hoy)
15/11	Entrega 4: app deployada, tests, documentación

✔️ Dudas → #consultas de Discord